Number Systems II

Contemporary Digital Electronic Systems — Chapter 3

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Chapter
2. Signed Numbers
3. Real Numbers
4. Fixed-Point Numbers
5. Floating-Point Numbers
6. Summary

Why This Chapter Matters

Chapter 2 covered only non-negative numbers. Real computers must handle negative values and represent fractions. This chapter introduces the signed representations, explains why two's complement dominates, and introduces fixed- and floating-point formats.

After this chapter, you can:

1. Convert between sign–magnitude, one's and two's complement.
2. Explain why two's complement dominates.
3. Represent a number in fixed-point and IEEE 754 single precision.

An Opening Question

What range of values can 4 bits represent in two's complement? Why is there one more negative value than positive?

We return to this at the end of the signed-number section.

Table of Contents

1 Why This Chapter

2 **Signed Numbers**

3 Real Numbers

4 Fixed-Point Numbers

5 Floating-Point Numbers

6 Summary

Unsigned and Signed-Magnitude

Unsigned

$V = \sum_{i=0}^{n-1} b_i 2^i$, range 0 to $2^n - 1$.
Cannot hold negative values.

Signed-magnitude

MSB is the sign (0 positive, 1 negative); the rest is the magnitude.

$$V_{\mathrm{SM}} = (-1)^{b_{n-1}} \sum_{i=0}^{n-2} b_i 2^i$$

Range: $-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$.

Drawbacks

- Two zeros: $+0 = 0000$, $-0 = 1000$.
- Adding numbers of opposite sign is not a simple addition.

Example: 1010

$(-1)^1 \times (0 \cdot 4 + 1 \cdot 2 + 0 \cdot 1) = -2$

Ones' Complement

- Negate by inverting every bit.
- Inverting twice restores the original.
- Range: $-(2^{n-1}-1)$ to $+(2^{n-1}-1)$.
- Still two zeros: 0000 and 1111.

$$V_{1\mathrm{C}} = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i + b_{n-1}$$

Example: 1010

$-8 + 2 + 1 = -5$

Val	1C	Val	1C
+7	0111	−0	1111
+6	0110	−1	1110
+5	0101	−2	1101
+4	0100	−3	1100
+3	0011	−4	1011
+2	0010	−5	1010
+1	0001	−6	1001
+0	0000	−7	1000

Two's Complement

- Negate by inverting every bit, then adding 1.
- Only one zero.
- Range: -2^{n-1} to $+(2^{n-1}-1)$: one more negative than positive.

$$V_{2\mathrm{C}} = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i$$

Example: 1010

$-8 + 2 = -6$

Val	2C	Val	2C
+7	0111	−1	1111
+6	0110	−2	1110
+5	0101	−3	1101
+4	0100	−4	1100
+3	0011	−5	1011
+2	0010	−6	1010
+1	0001	−7	1001
+0	0000	−8	1000

Finding a Two's Complement

Subtract from 2^n

Six-bit representation of -9:
$2^6 - 9 = 1000000_2 - 001001_2 = 110111_2$

From ones' complement: add 1

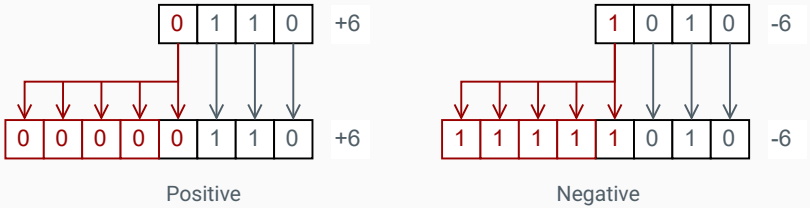
$1101_{1C} + 1 = 1110_{2C}$, and $1110 = -8 + 4 + 2 = -2$.

Quick method

1. Start at the LSB; copy all zeros.
2. Copy the first 1.
3. Invert all remaining bits.

Negating 0011 1000 gives 1100 1000.

Sign Extension



- To widen a two's-complement number, copy the sign bit into the new high-order bits.
- The value is unchanged. (Unsigned numbers are padded with 0s instead.)

© Muním Zabidi

Comparing the Systems (4 bits)

Bits	Unsig.	SM	1C	2C
1111	15	−7	−0	−1
1110	14	−6	−1	−2
1101	13	−5	−2	−3
1100	12	−4	−3	−4
1011	11	−3	−4	−5
1010	10	−2	−5	−6
1001	9	−1	−6	−7
1000	8	−0	−7	−8
0000–0111	0–7	0–7	0–7	0–7

	SM	1C	2C
Smallest	$-(2^{n-1}-1)$	$-(2^{n-1}-1)$	-2^{n-1}
Largest	$2^{n-1}-1$	$2^{n-1}-1$	$2^{n-1}-1$
Zeros	2	2	1

- MSB = 0: every system gives the same value.
- A byte: unsigned 0 to 255; two's complement -128 to $+127$.

Back to the Opening Question

Range of 4-bit two's complement

With $n = 4$ the range is -2^{n-1} to $+(2^{n-1} - 1)$: -8 to $+7$, sixteen values in all.

Zero uses one pattern (0000) and the sign bit splits the remaining 15 patterns unevenly: seven positive (0001 to 0111) and eight negative (1000 to 1111). The extra one is $1000 = -8$, which has no positive counterpart in 4 bits.

Preview: Addition and Overflow (Chapter 9)

- One *n*-bit adder adds signed and unsigned numbers alike; subtract by adding the negation.
- Overflow: the true result does not fit in *n* bits. It happens only when adding two numbers of the *same sign* and the result's sign differs.
- Equivalent test: carry *into* the sign bit $\neq$ carry *out of* it.

$5 + 4$	$5 + 5$	$-2 + -3$
$0101 + 0100 = 1001$ reads as -7: overflow	$0101 + 0101 = 1010$ reads as -6: overflow	$1110 + 1101 = 1\,1011$ $= -5$, carry-out 1 but no overflow

A result outside the range -8 to $+7$ (4 bits) cannot be represented, so the hardware must raise an overflow flag. Chapter 9 covers addition in detail.

Table of Contents

1 Why This Chapter

2 Signed Numbers

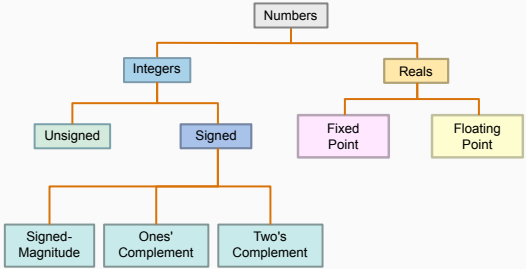
3 **Real Numbers**

4 Fixed-Point Numbers

5 Floating-Point Numbers

6 Summary

Representing Real Numbers



System	Format	Note
Rational	$\pm p/q$	Difficult to work with
Fixed-point	$\pm i.f$	Low precision
Floating-point	$\pm m \cdot b^e$	Most common for reals

- No computer system can represent all real numbers.
- We cover fixed-point and floating-point.

© Muním Zabidi

Table of Contents

1. Why This Chapter
2. Signed Numbers
3. Real Numbers
4. **Fixed-Point Numbers**
5. Floating-Point Numbers
6. Summary

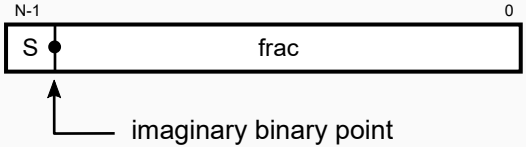
- A fixed number of bits for the integer part and the fraction. The binary point is *not stored*; the designer remembers where it is.

$$x = \frac{-b_{N-1}2^{N-1} + b_{N-2}2^{N-2} + \cdots + b_0 2^0}{2^f}$$

Example: 0101.1100_2 (8 bits, $f = 4$)

$0 + 4 + 0 + 1 + 0.5 + 0.25 + 0 + 0 = 5.75_{10}$. Stored as 01011100_2.

- Q*f*: 1 sign bit and *f* fractional bits. Q7 = 8 bits; Q15 = 16 bits.
- Range $[-1,\ 1-2^{-f}]$. The value $+1$ cannot be represented.
- Common in DSP, embedded systems and TinyML: integer hardware does fractional maths.



Q7 limits

Min $10000000 = -1.0$; max $01111111 = 0.9921875$; resolution $2^{-7} = 0.0078125$.

© Munim Zabidi

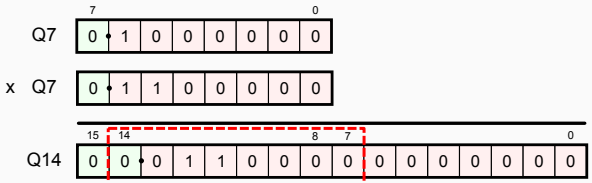
Example: 0.8 in Q7 and Q15

$0.8_{10} = 0.110011001100\ldots_2$ (not exact)

- Q7: $0.1100110_2 = 0.796875$; error about 0.39%.
- Q15: $0.110011001100110_2 = 0.79998779296875$; error about 0.0015%.

- More bits give better accuracy but more hardware cost and latency.
- Choose the minimum precision that meets the quality requirement.

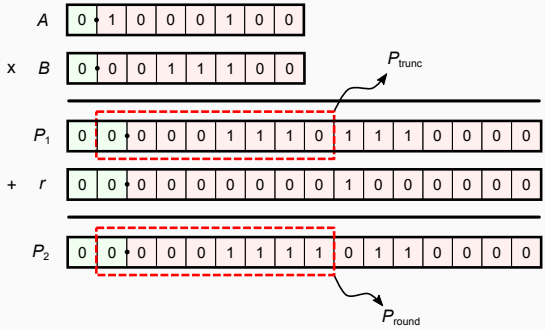
- Q-format uses the same integer circuits; only the interpretation differs.
- Adding two m-bit numbers gives an $(m+1)$-bit sum. Check the overflow flag.
- Multiplying m-bit by n-bit gives an $(m+n)$-bit product.



0.5×0.75: $01000000 \times 01100000 = 0001\,1000\,0000\,0000$ (two sign bits)

- Systems keep one data width, so the product is cut back to Q7.

Truncation versus Rounding



- Example: 0.53125×0.21875; exact $= 0.1162109375$.
- Truncation drops low bits: 0.109375, error 5.9%.
- Rounding: add $r = 2^6$ first, then truncate: 0.1171875, error 0.8%.

Table of Contents

1 Why This Chapter

2 Signed Numbers

3 Real Numbers

4 Fixed-Point Numbers

5 **Floating-Point Numbers**

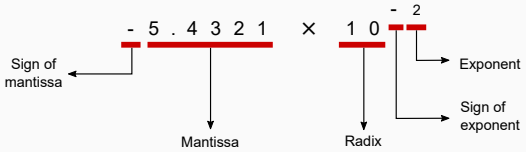
6 Summary

Floating-Point: Significand and Exponent

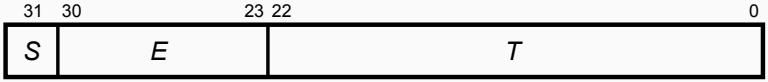
- $X = m \times b^e$: the radix point floats as the exponent changes.
- Huge dynamic range, but costs more hardware and energy than fixed-point.
- Normalized binary form has a leading 1 before the point, e.g.

$$5.25_{10} = 101.01_2 = 1.0101_2 \times 2^2$$

IEEE 754 does not store the leading 1: the implicit 1 gives one extra bit of precision.



IEEE 754 Single Precision (FP32)



Normal numbers ($1 \leq E \leq 254$)

$$X = (-1)^S \times \left(1 + \frac{T}{2^{23}}\right) \times 2^{E-127}$$

- S: sign, 1 bit. E: exponent, 8 bits, bias 127. T: trailing significand, 23 bits.
- 24-bit precision, about 7 significant decimal digits.

FP32 Examples

Decode `1 10000001 11000...0`

$S = 1, E = 129, m = 1.11_2 = 1.75$
$X = -1.75 \times 2^{129-127} = -7$

Encode 17.0

$17 = 10001_2 = 1.0001_2 \times 2^4$
$S = 0, E = 127 + 4 = 131 = 10000011_2$
`0 10000011 0001000...0`

Special Values and Subnormals

Value	E	T	Hex
$+0$	0	0	0x00000000
-0	0	0	0x80000000
1.0	127	0	0x3F800000
$+\infty$	255	0	0x7F800000
$-\infty$	255	0	0xFF800000
NaN	255	$\neq 0$	0x7FFFFFFF
Largest normal	254	all 1s	0x7F7FFFFF
Smallest normal	1	0	0x00800000

- Subnormal: $E = 0$, $T \neq 0$; leading bit 0; fills the gap between 0 and 2^{-126} (gradual underflow).
- Exceptions: invalid, divide by zero, overflow, underflow, inexact.

Other Floating-Point Formats

Format	Bytes	Exponent bits	Fraction bits	Precision (bits)
FP16 (half)	2	5	10	11
FP32 (single)	4	8	23	24
FP64 (double)	8	11	52	53
FP128 (quad)	16	15	112	113
BF16 (bfloat16)	2	8	7	8
FP8 E4M3	1	4	3	4
FP8 E5M2	1	5	2	3

- BF16 and FP8 are used in machine learning: same range as FP32 (BF16) or a range/precision trade-off (FP8).

© Muním Zabidi

Table of Contents

1 Why This Chapter

2 Signed Numbers

3 Real Numbers

4 Fixed-Point Numbers

5 Floating-Point Numbers

6 **Summary**

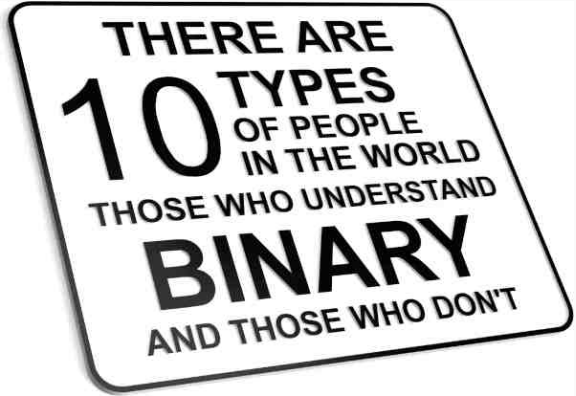
Summary

- Two's complement dominates: a single representation of zero and a simple positional formula with sign-bit weight -2^{n-1}.
- Negate by inverting every bit and adding 1.
- Fixed-point trades range for simplicity; floating-point trades hardware for dynamic range.
- IEEE 754 single precision: sign (1), biased exponent (8), trailing significand (23).

Looking ahead

Numbers in, numbers out. Chapter 4 introduces the logic gates that carry out every operation on those numbers.

One Last Thing



Hint: read the 10 as a binary number.